

Practicals: Botnet Detection

At the end of each workshop, each student should submit a report of the solutions of the steps, following the specific directives as illustrated in each workshop document. Some steps do not have outputs. The steps that have outputs are marked as a whole by the end of the trimester.

Most of the practicals use Google Colab with Python, which is an online deep-learning platform. Links to the colab notebooks will be provided as reference. Refer to the colab notebooks if you are not sure how to do the steps. You can make a copy of the colab notebook to edit and run for your practicals.

In the report, include the title of the practical, the steps performed, and the output/result of each of the steps completed.

Questions:

- 1) **Use machine learning algorithms to differentiate between normal traffic and botnet traffic**
- 2) **Analyze data to find missing values and visualize them**
- 3) **Select a set of evaluation criteria (Accuracy, Precision, ROC, AUC, Recall or F-measure)**
- 4) **Compare different models**

The practical is based on the following Github code:

<https://github.com/ShehzadaAlam/Botnet-Detection/blob/master/Botnet%20Detection.ipynb>

It uses machine learning to differentiate between different normal and attack traffic. The dataset to be used is the CTU-13 dataset which can be downloaded through the given link: <https://mcfp.Weebly.com/the-ctu-13-dataset-a-labeled-dataset-with-botnet-normal-and-background-traffic.html>.

Link to colab:

<https://colab.research.google.com/drive/15nFlwwueZjfp5MrVyb9EfuNk4cwCXZE?usp=sharing>

The code and data are based on the following github. Download the following data from here:

<https://drive.google.com/drive/folders/14SLU9--GnB8hbdVyZ0rUtvb15TWOExkn?usp=sharing>

1. capture20110810.binetflow.2format

CTU-13 datasets

The dataset is a botnet traffic dataset that was built in the year 2011 by the CTU University located in the Czech Republic. It is a labelled dataset that contains instances of botnet, normal and background traffic. The dataset is a mixture of traffic and consists of 13 scenarios where each instance was created by keeping in mind a different malware.

The normal traffic mainly involves users performing normal activities such as accessing Email records and surfing the web, while background traffic is generated to obscure the presence of botnet traffic. Each scenario was caught in a pcap (packet capture) file that involved all three variants of traffic. The datasets contain a total of 15 features. The CTU dataset scenario 1 has a total of 2824636 records out of which 97.5% traffic is background traffic, botnet traffic is 1.5% and normal traffic is 1%. It was also noticed that 80.4% of the traffic uses UDP protocol and 18% uses TCP protocol. The traffic was mainly bi-directional around 77.6% which travelled from source to destination. The 15 features of the dataset are shown below.

Table 3-8: Features in CTU-13 dataset Kirubavathi and Anitha (2018)

Feature Number	Features Name	Feature Description
1	Start Time	Represents the Start time of the attack and uses the format 2011/08/10 12:30:13.516854
2	Dur	Represents the duration of attack and is represented in seconds
3	Proto	Consists of 15 different protocols namely 'tcp', 'udp',
4	SrcAddr	Source IP address
5	Sport	Source Port address where the traffic originated from
6	Dir	Direction of the traffic represented as '->', '?>'
7	DstAddr	Destination IP Address
8	Dport	Destination Port address where traffic was originated
9	State	Represents the state of transactions according to the protocol and has 231 unique values
10	Stos	Source type of service field typical 0,1,2,3, 192, NaaN
11	dTos	Destination type of service typical 0,1,2,3, NaaN
12	TotPkts	It represents a total transaction packet count and it ranges from 1 to 2686731
13	TotBytes	Total transaction bytes ranging from 60 to 2689640464
14	SrcBytes	Total transaction bytes from source to destination ranging from 0 to 2635366235
15	Label	Three unique traffic labels namely background

		botnet and normal,
--	--	--------------------

Guidelines To follow:

Step 1: Load the necessary packages and libraries that you will be needing. Some sample library imports have been shown below:

```
#importing all necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')
```

Step 2 : Load the Dataset using `pd.read_csv` function, make sure to add the correct path to the data.

```
# Mounting my google drive here in google colab using
# the following code
# ** important. For security reason, never allow other
# colab to have access to your folder
from google.colab import drive
drive.mount('/content/drive')

# Adding the path to train and test data
# Goto "MyDrive/Coab Notebooks" folder on your Google drive.
# Inside the folder, create folder structure
# "Datasets/Botnet_data/capture20110810.binetflow.2format"
# Upload the downloaded csv data files to the folder.
Path_to_train_data = "/content/drive/MyDrive/Colab
Notebooks/Datasets/capture20110810.binetflow.2format"
```

Step 3 : Read botnet data:

```
dataset = dataset[dataset['Label'].str.contains('Botnet')]
```

Display the first 5 rows of the dataset to see whether the information is correctly extracted.

```
dataset.head() #print 5 rows
```

Check the result if it looks like this.

	SrcAddr	DstAddr	Proto	Sport	Dport	State	sTos	dTos	SrcWin	DstWin	...	SAppBytes	DAppBytes	Dur	TotPkts	TotBytes	TotAppByte	Rate	SrcRate	Ds
0	94.44.127.113	147.32.84.59	tcp	1577	6881	RST	0.0	0.0	131070.0	0.0	...	0	0	1.026539	4	276	0	2.922441	0.974147	1.5
1	94.44.127.113	147.32.84.59	tcp	1577	6881	RST	0.0	0.0	131070.0	0.0	...	0	0	1.009595	4	276	0	2.971488	0.990496	1.5
2	147.32.86.89	77.75.73.33	tcp	4768	80	RST	0.0	0.0	65535.0	6886.0	...	0	0	3.056586	3	182	0	0.654325	0.327162	0.0
3	147.32.86.89	77.75.73.33	tcp	4788	80	RST	0.0	0.0	65535.0	6897.0	...	0	0	3.111769	3	182	0	0.642721	0.321361	0.0
4	147.32.86.89	77.75.73.33	tcp	4850	80	RST	0.0	0.0	65535.0	6809.0	...	0	0	3.083411	3	182	0	0.648632	0.324316	0.0

5 rows x 33 columns

You can also use `dataset.info()` to print out a brief description of different attributes present in the dataset, their data types and the number of rows.

`dataset.description()` can also be used to get basic statistics about the dataset which include mean and standard deviation.

Note : Make sure to use the same variables as the ones set by you.

Step 4: Preprocess the datasets to get rid of missing values.

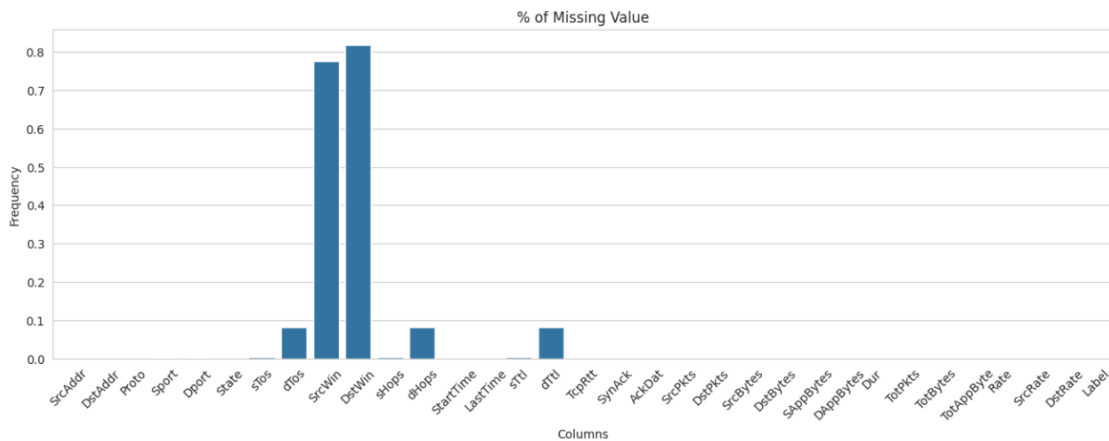
First, we visualize the attributes that have the most missing values. This will provide a brief description about the attributes. The visualization of the missing values can be done using a bar graph. This can be performed by using the seaborn library.

```
# Visualizing the Missing value
plt.figure(figsize=(16,5))

# using the seaborn library it is possible to set styles
sns.set_style('whitegrid')

# add title to graph
plt.title('% of Missing Values')

# getting the columns that has null values and the number of
# null values for each column
sns.barplot(x=dataset.isnull().mean().index,
y=dataset.isnull().mean().values)
plt.xlabel('Columns') # assigning x label
plt.xticks(rotation=45)
plt.ylabel('Frequency') # assigning y label
plt.show() #display graph
```



Note: Make sure to specify the figure size using the function `plt.figure(figsize = (a,b))` , where a, b are some arbitrary values. Add a xlabel, labels and titles so that the graph interpretation becomes easier. Additionally graph styles can be added using `sb.set_style('Put the style name here')`. Styling makes the graph more appealing.

Note : Make sure to add comments after each code to keep track of operations being computed and make sure to use the right variable names. Make sure to print the dataset to verify the changes made.

We now remove all attributes or columns having more than 30% of missing values:

```
# Removed all the columns with missing value > 30%
dataset = dataset.loc[:, dataset.isnull().mean() < 0.3]
```

Step 5: Preprocessing step continues.

We change some of the data types. After which certain attributes will be dropped from the dataset. A new attribute will also be added. Each of these are individually illustrated below.

Change the datatype of the following attributes:

1. Change the data type of Proto to Category
2. Change the data type of Sport to Category
3. Change the data type of Dport to Category
4. Change the data type of State to Category
5. Change the data type of StartTime to datetime64
6. Change the data type of LastTime to datetime64

```
# Changing the datatype of the columns
dataset =
dataset.astype({"Proto": 'category', "Sport": 'category', "Dport": 'category', "State": 'category', 'StartTime': 'datetime64[s]', 'LastTime': 'datetime64[s]'})
```

Derive a new attribute/feature called **duration** using two already available attributes: StartTime and LastTime. Duration will be computed by taking the difference between Lastime. StartTime should be in seconds.

Note: Make sure to measure the absolute values of the difference, to mitigate any negative calculated time.

```
# Getting duration from the columns 'LastTime' and 'StartTime'
dataset['duration'] = abs(dataset['LastTime'].dt.second -
dataset['StartTime'].dt.second)
```

Drop the following attributes:

1. SrcAddr
2. DstAddr
3. LastTime
4. Start Time

We are dropping the columns SrcAddr and DstAddr since they contain unique ip addresses.

Attributes LastTime and StartTime are not needed, we need these two for the computation of the feature called duration, which is shown earlier. The sample code for dropping attributes is shown below:

```
# Drop the selected columns
dataset.drop(columns=['SrcAddr', 'DstAddr', 'LastTime', 'StartTime'], in
place=True)
```

Note: Make sure to put the correct attribute names otherwise the code will show errors.

Step 6: Training and evaluating a classification model: Random Forest Classifier (RTC)

Training will be done using Random Forest Classifier (RTC) using Decision Tree Classifier as the basis classifier. RTC is an ensemble classifier that combines weak classifiers in order to improve the performance.

First, extract only categorical columns by removing all columns of data type int64 and Float64.

```
# Extract only categorical columns
categorical_columns = dataset.select_dtypes(exclude=['int64',
'float64']).columns.values
```

To Understand what the working of the code visit : https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.select_dtypes.html

We now separate out the datasets into two parts: X and Y. X stores the features values. Y holds all the labels.

```
# Separating out the features from the label

# the features
X = dataset.loc[:, dataset.columns != 'Label']

# the labels
y = dataset.loc[:, dataset.columns == 'Label']
```

We now convert the labels, which are categorical values, into binary values. To do this, we use the `pd.get_dummies()` function.

```
# Convert a categorical column into N binary columns
temp = pd.get_dummies(y)
```

Check temp to see what happen:

```
# print temp
temp
```

Now, convert it into a value array. This will convert the data into a suitable format for training:

```
# Get the values of labels.
y = temp.values
```

To Understand what the working of the code visit :

https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html

We now preprocess x data as well.

Drop the following attributes from x:

1. Proto
2. Sport
3. Dport
4. State

```
# List the column names to be removed from X
columns = ["Proto", "Sport", "Dport", "State"]

# Drop columns
X = X.drop(columns, axis=1)
```

Print the feature set X to verify.

We now perform min-max normalization on the feature set X to normalize the values:

```
# min-max normalization: normalizing X
```

```
X = (X - X.mean()) / (X.max() - X.min())

# printing to see normalization
X
```

Drop columns having NaN values:

```
# Drop columns having NaN values
columns = ["sTos", "sHops", "sTtl"]
X = X.drop(columns, axis=1)
```

We now get the values from the preprocessed data. This will convert the data into a suitable format for training:

```
# We now extract the values from X
X = X.values

# Print X to check
X
```

We now split the data into training and test data. We will use an 80-to-20% ratio for training and testing:

```
# import the library for data split
from sklearn.model_selection import train_test_split

# split the data into training and test data
x_train, x_test, y_train, y_test = train_test_split(X,y,
test_size=0.2, random_state = 42)
```

We will now train a DT classifier and calculate its performance:

```
# import libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# create DT classifier model
descision_tree_model = DecisionTreeClassifier()

# Train the model
descision_tree_model.fit(x_train,y_train)
```

Perform prediction using the test data:

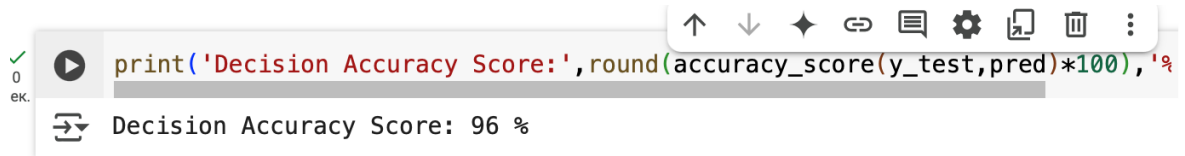
```
pred = descision_tree_model.predict(x_test)

# print to check
```


pred

Print out the performance result:

```
print('Decision Accuracy  
Score:',round(accuracy_score(y_test,pred)*100),'%') #getting the  
accuracy
```



A screenshot of a Jupyter Notebook interface. The top part shows a code cell with the following Python code: `print('Decision Accuracy Score:',round(accuracy_score(y_test,pred)*100),'%')`. Below the code cell, the output is displayed: `Decision Accuracy Score: 96 %`. The interface includes standard Jupyter icons for running, undo, redo, and other actions.

Step 7: Repeat training and testing for Random Forest Classifier. Get the performance results and compare their performance.

```
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score  
# Use a reduced dataset for speed  
x_train_sample = x_train[:1000]  
y_train_sample = y_train[:1000]  
x_test_sample = x_test[:200]  
y_test_sample = y_test[:200]  
  
# Train the model  
rf_model = RandomForestClassifier()  
rf_model.fit(x_train_sample, y_train_sample)  
  
# Predicted  
rf_pred = rf_model.predict(x_test_sample)  
  
# Get accuracy  
print("Random Forest Accuracy Score:",  
round(accuracy_score(y_test_sample, rf_pred) * 100), "%")
```



A screenshot of a Jupyter Notebook interface showing the output of the Random Forest Classifier. The output is displayed as a text cell: `Random Forest Accuracy Score: 89 %`. The interface includes standard Jupyter icons for running, undo, redo, and other actions.